

Machine Learning for Software Engineering

ISW2 project a.y. 2023/2024

Stefano Belli, matricola 0350116

Università degli Studi di Roma "Tor Vergata"

Agenda

1. Introduzione

2. Scopo

3. Metodologia

- Recupero informazioni su release e tickets da Jira
- Ciclo di vita del difetto
- Utilizzo della tecnica proportion
- Metodi di utilizzo di proportion
- Costruzione del dataset: metriche
- Costruzione del dataset: buggyness e snoring
- Classificatori e tecniche considerate
- Evaluation con walk forward

4. Risultati

- Risultati per BookKeeper
- Risultati per Storm

5. Conclusioni e minacce alla validità

6. Links

Introduzione

- Nei grandi progetti software, il collaudo è cruciale: avere una buona test suite aiuta a prevenire malfunzionamenti spiacevoli per l'utente, permettendo il rilevamento del difetto prima della release.
- Tuttavia, eseguire numerosi test è dispendioso sia in termini di tempo d'esecuzione dei test sia per quanto riguarda la loro progettazione e implementazione: per quali unità conviene focalizzarsi?
- Per aiutarci a rispondere a questa domanda, possiamo sfruttare un modello di machine learning che date informazioni sul codice sorgente, predica se l'unità in esame sia "buggy" o meno (e quindi, ci aiuti a decidere se generare dei test case per quest'ultima).

Scopo

- Si vogliono confrontare le prestazioni di tre classificatori (Naive Bayes, IBk, Random Forest) con e senza tecniche di feature selection, sampling e cost sensitivity.
 - ▶ Prediction della buggyness di una classe Java
- Per la costruzione del dataset (e quindi la comparazione dei classificatori) vengono sfruttati i progetti open source Apache BookKeeper e Apache Storm.

Recupero informazioni su release e tickets da Jira

Dal bug tracker Jira, utilizzato dalla Apache Software Foundation, vengono scaricati, tramite la sua REST API, informazioni riguardo alle release e ai ticket che siano di tipo *"bug"*, il cui stato sia *"closed"* o *"resolved"*, e la risoluzione *"fixed"*

Le informazioni (in particolare le date) su questi tipi di ticket serviranno poi per determinare quali classi sono buggate, in quali release.

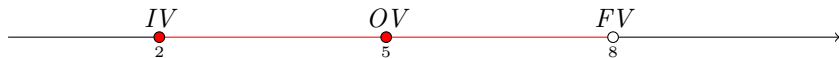
In Jira, nell'oggetto JSON della risposta, per ogni ticket, sono di particolare interesse questi campi (che ci permetteranno di capire le release affette dal bug):

- L'array *fields.version* che contiene le informazioni sulle release affette dal bug (affected versions)
- La stringa *fields.created* che è la data di apertura del ticket
- La stringa *fields.resolutiondate* che è la data di risoluzione del bug

Ciclo di vita del difetto

Al fine di determinare le versioni affette dal bug, definiamo:

- L' **injected version** (IV) che è il numero di versione in cui è stato introdotto il bug
- L' **opening version** (OV) che è il numero di versione in cui è stato aperto il ticket relativo al bug (che è stato quindi rilevato)
- La **fixed version** (FV) che è il numero di versione in cui è stato risolto il bug



Tutte e 3 le IV , OV e FV permettono di individuare il "ciclo di vita" completo del bug, che è presente da IV (incluso) fino a FV (escluso).

Problema

Le API di Jira, forniscono sempre informazioni riguardo alla creazione del ticket e alla sua risoluzione (permettendo di determinare, rispettivamente: opening e fixed version), ma non sempre rispetto alle affected versions, non permettendo sempre di individuare la injected version!

Utilizzo della tecnica proportion

Se l'injected version non è nota, per un certo bug possiamo sfruttare il **proportion** per stimare l' IV :

$$p = \frac{FV - IV}{FV - OV} \Rightarrow IV = FV - (FV - OV) \cdot p$$

Applicazione

A partire dai bug la cui IV è nota, calcoliamo p (prima formula). Poi, avendo appena calcolato la costante di proporzionalità p , per il bug di cui non si conosce IV , si applica la seconda formula, in modo da determinarla.

Metodi di utilizzo di proportion

Viene utilizzato proportion con le tecniche:

- **Cold start:** se il numero di tickets utilizzabili ai fini del calcolo di p è minore di 5, si calcola la media di p sui ticket per ognuno dei seguenti progetti Apache: Avro, OpenJPA, Syncope, Tajo e ZooKeeper. Infine si effettua la mediana su queste 5 medie, ottenendo il valore di p finale.
- **Incremental:** se il numero di tickets utilizzabili ai fini del calcolo di p è maggiore o uguale a 5: p è calcolato effettuando la media delle proporzionalità p di tutti i bug precedenti (che hanno IV) rispetto a quello considerato attualmente, di cui bisogna stimare IV .

Costruzione del dataset: metriche

- Viene usato JGit al fine di poter manipolare un repository Git direttamente tramite metodi Java: possiamo collezionare informazioni su commit e quindi anche sul codice sorgente (esclusivamente classi Java) del repo di interesse.
- Dalle informazioni ricavate da Git, si possono calcolare delle metriche di interesse su un certo file Java che verranno poi utilizzate dal classificatore ai fini della predizione

Le metriche di interesse scelte sono le seguenti:

Metrica	Descrizione
LOC	Numero di linee di codice
LOC added	Somma di LOC aggiunte sulle revisioni
Avg. LOC added	Media di LOC aggiunte sulle revisioni
Max. LOC added	Numero massimo di LOC aggiunte tra le revisioni
Churn	Somma di LOC aggiunte e rimosse sulle revisioni
Avg. churn	Media di LOC aggiunte e rimosse sulle revisioni
Max. churn	Numero massimo di LOC aggiunte e rimosse tra le revisioni
Avg. chg. set	Media di file cambiati insieme sulle revisioni
Max. chg. set	Numero massimo di file cambiati insieme sulle revisioni
Num. of authors	Numero di autori
Num. of revs	Numero di revisioni

Costruzione del dataset: buggyness e snoring

- Le informazioni ottenute da Jira vengono "collegate" a quelle ricavate da Git (ticket e release vengono associati ai commit).
- Il dataset, oltre ad essere composto dalle metriche di interesse appena elencate, ha nella prima e seconda colonna rispettivamente la versione del progetto Apache in esame, il path al file Java interessato, nel mezzo le metriche, e come ultima colonna la buggyness.
- La **buggyness** (variabile binaria che può assumere valore "YES" o "NO" e che il classificatore deve predire) viene determinata grazie al collegamento delle informazioni ricavate tra Jira e Git.
 - ▶ Una classe Java è *buggy* se la versione v in cui ricade è compresa nell'intervallo $IV \leq v < FV$

Snoring

Al fine di ridurre l'effetto dello **snoring**, ovvero bug dormienti non rilevati, sono stati eliminati tutti i dati relativi alla seconda metà delle releases. Dare in pasto al classificatore dati affetti da snoring può comprometterne le prestazioni.

Classificatori e tecniche considerate

Sono stati utilizzati 3 classificatori diversi...

- Random forest
- Naive Bayes
- IBk

...ognuno con le seguenti tecniche...

Feature selection

- Nessuna tecnica di feature selection
- Best first backward

Sampling

- Nessuna tecnica di sampling
- SMOTE
- Undersampling

Cost sensitivity

- Nessuna tecnica di cost sensitivity
- Sensitive Learning

...combinare come segue:

Nessuna tecnica utilizzata
Solo sensitive learning
Solo best first backward
Solo undersampling
Solo SMOTE
Best first backward e sens. learning
Best first backward e undersampling
Best first backward e SMOTE

La valutazione viene effettuata tramite l'ausilio 4 metriche: **F1-score** (media armonica tra precision e recall, il cui valore sta nel range $[0, 1]$: è 1 se lo sono sia precision che recall, è 0 alla condizione che lo siano sia precision che recall), **Kappa**, **AUC**, **NPofB20** (percentuale di bug che possono essere trovati analizzando il top 20% di LOC. Il ranking, per definire il "*top 20% di LOC*" è effettuato sulla prob. pred. di buggyness diviso la size della classe).

Ne viene valutato, inoltre, il **costo**: $CFN \cdot FN + CFP \cdot FP$ con $CFN = 10$, $CFP = 1$

Evaluation con walk forward

Lo strumento utilizzato per valutare le prestazioni dei classificatori (con o senza tecniche) è una tecnica di tipo time-series: walk forward.

- Nella figura sottostante è presente un esempio di esecuzione dell'evaluation: ogni cella, rappresenta i dati relativi alla release i -esima (dove i è la colonna di appartenenza) e la colorazione indica la split che viene attuata alla j -esima iterazione (dove j è la riga)
- Nel nostro caso, la prima run, non viene effettuata a prescindere dato che non abbiamo un training set con cui addestrare un classificatore e quindi viene ritenuta inutile.
- Le run per BookKeeper partono dalla run 3 (2 training rels. + 1 testing rel), mentre le run per Storm partono dalla run 4 (3 training rels. + 1 testing rel.)
 - ▶ Essendo una tecnica time-series, i dati disponibili fino alle release di training non rivelano alcuna classe buggy, impedendo l'addestramento del classificatore

Run	Part				
	1	2	3	4	5
1	Testing				
2	Training	Testing			
3	Training	Training	Testing		
4	Training	Training	Training	Testing	
5	Training	Training	Training	Training	Testing

Il **training set** deve riflettere l'utilizzo reale del classificatore, può essere affetto da snoring: non utilizziamo i dati del futuro per determinare la buggyness (la tecnica è time-series), al contrario, per il **testing set** vengono utilizzati tutti i dati a nostra disposizione, deve essere il più accurato possibile e non deve essere affetto da snoring (idealmente)

Risultati per BookKeeper: F1-score

Tra tutte le soluzioni proposte, **IBk no feature selection**, si **SMOTE**, no cost sensitivity, ha l'**F1-score migliore**.

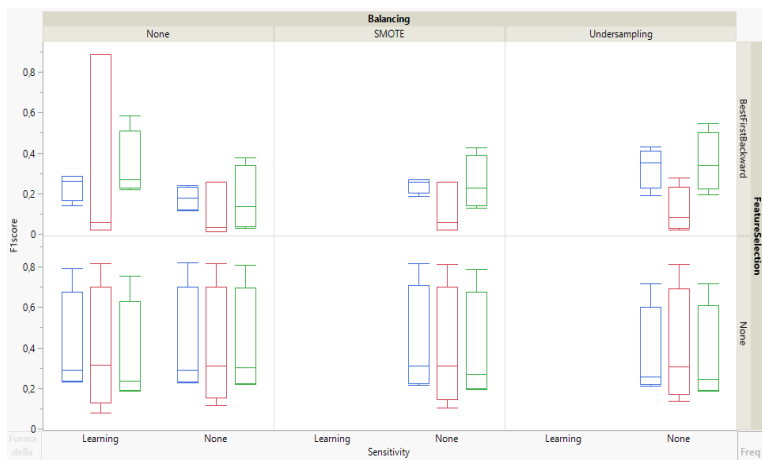


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per BookKeeper: AUC

Naive Bayes sembra essere quello che ha i valori massimi più alti, ma anche i valori minimi e mediane più bassi. Per quanto riguarda IBk, la mediana è decisamente più elevata rispetto agli altri due classificatori, con tutte le combinazioni delle tecniche. **Random forest, no feature selection, si SMOTE, no cost sensitivity, ha l'AUC migliore.**

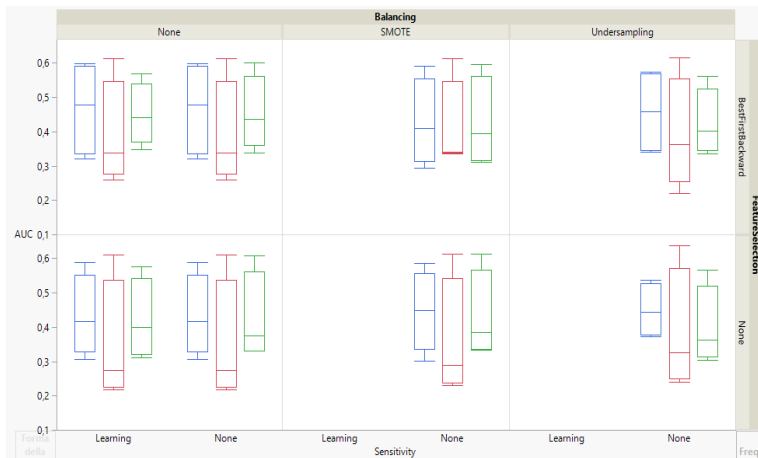


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per BookKeeper: NPofB20

In generale, l'applicazione di feature selection peggiora NPofB20 per tutti i classificatori e le possibili combinazioni di balancing e cost sensitivity (tutti i 3 riquadri sopra). **IBk, no feature selection, si SMOTE, no cost sensitivity, ha l' NPofB20 migliore** (nonostante altri classificatori riescono ad ottenere risultati simili in termini di massimo, riesce ad avere minimo e mediana più alti).



Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per BookKeeper: costi

Risulta chiaro che l'applicazione di feature selection incrementa notevolmente il costo. **Random forest senza l'applicazione di alcuna tecnica è il classificatore con il costo più basso**

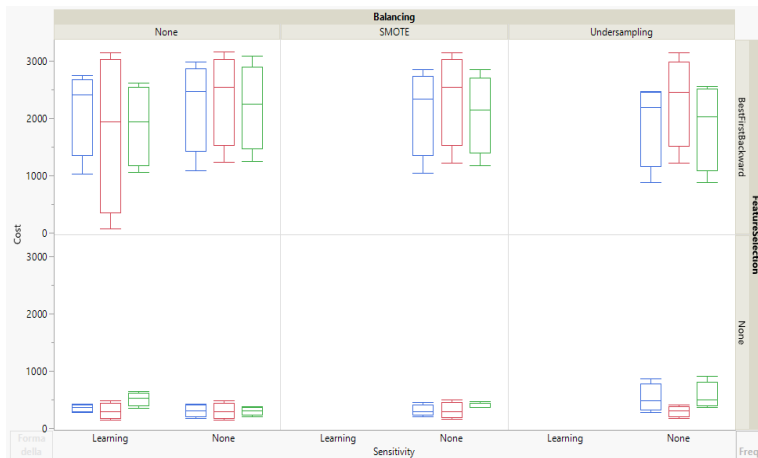


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per Storm: F1-score

Tutti i classificatori, di tutte le combinazioni presentano, più o meno, lo stesso minimo. L'applicazione di feature selection sembra migliorare notevolmente il massimo assunto dai classificatori. In combinazione al feature selection, utilizzare sensitive learning migliora notevolmente la metrica F1-score, e quello con punteggio più alto è IBk o Random forest. A parità di minimo, **Random forest, si feature selection, no balancing, si sens. learning, risulta essere quello con punteggio F1 più alto**, ma con le stesse identiche tecniche applicate, anche IBk non è male: viene scelto RF solo perchè ha massimo e mediana più alti.

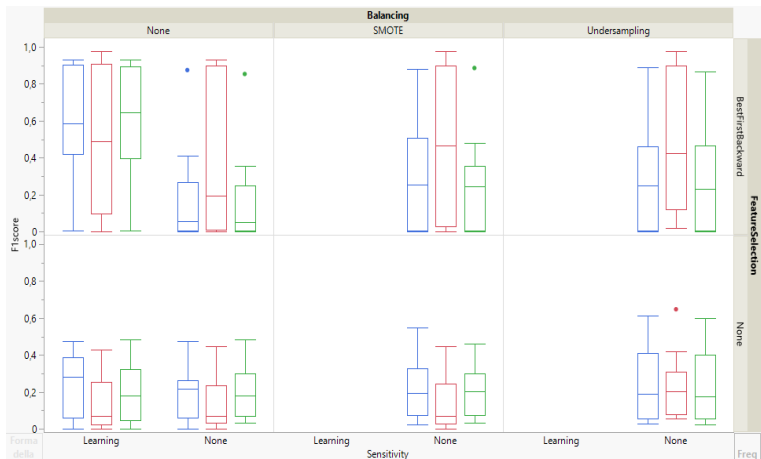


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per Storm: Kappa

Il sensitive learning si riconferma una tecnica molto efficace, in particolare con IBk.

Con un outlier > 0.4 , **IBk, si feature selection, no balancing, si sensitive learning** è la combinazione che **fornisce kappa migliore**. Si distingue anche la stessa configurazione, ma senza feature selection (di fatto, molto simili).

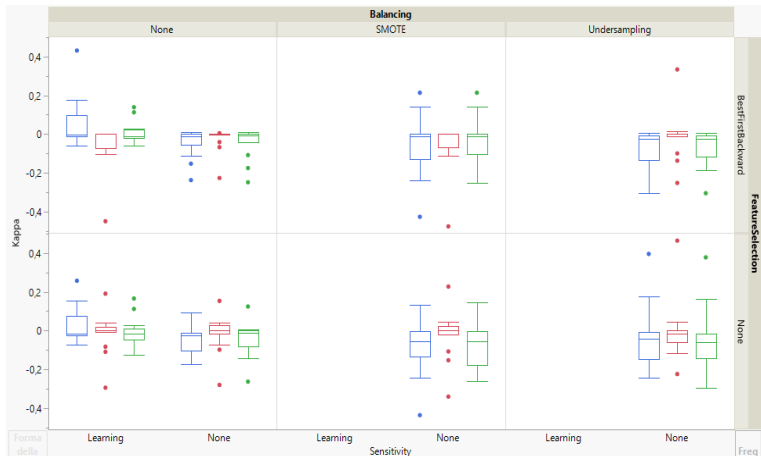


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per Storm: AUC

Per quasi tutte le combinazioni, IBk risulta essere il migliore in termini di AUC.

Tutti i box plot riguardanti AUC di IBk sono a loro volta simili, ad eccezione per l'undersampling.

Scelto IBk e scartate le configurazioni che hanno come balancing l'undersampling, si possono anche scartare quelle che non hanno applicato feature selection (i minimi sono gli stessi, i massimi più bassi, rispetto alle configurazioni che applicano feature selection).

IBk, si feature selection, no balancing, si sensitive learning, risulta essere quello con AUC migliore

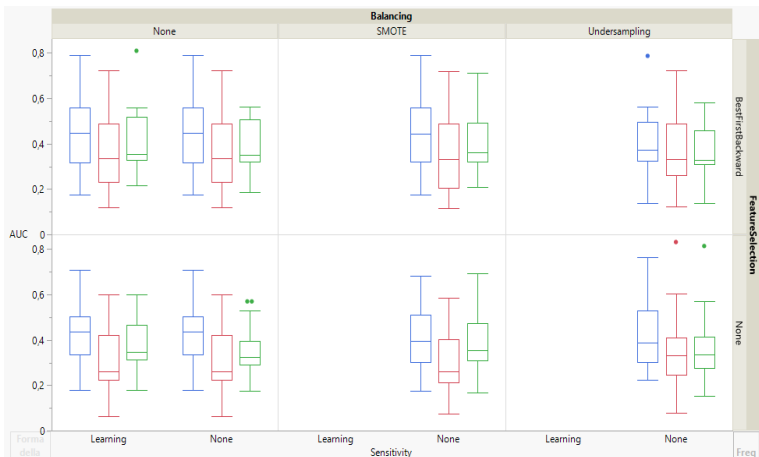


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per Storm: NPofB20

Random forest, si feature selection, no balancing, no sens. learning ha l'NPofB20 migliore

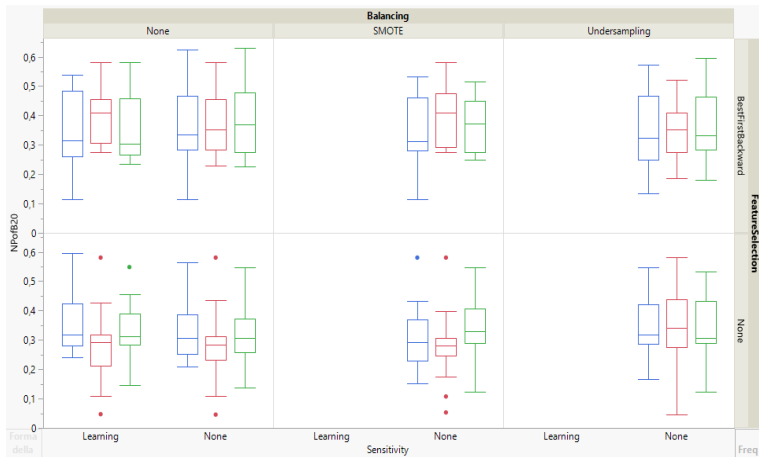


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Risultati per Storm: costi

IBk, no feature selection, no balancing, si sens. learning ha il costo migliore (tendenzialmente più basso)

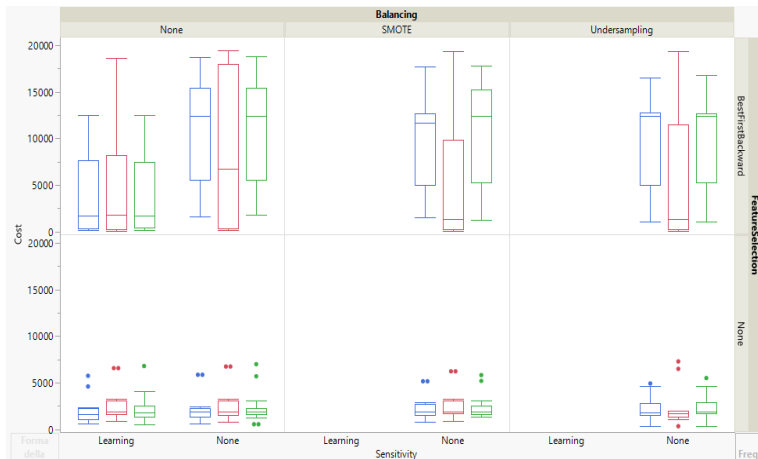


Figura: Il box plot verde è per Random forest, rosso per Naive Bayes, blu per IBk

Conclusioni e minacce alla validità

Conclusioni

- Non esiste una scelta migliore dell'altra, tuttavia, possiamo affermare che per BookKeeper una scelta non da fare è sicuramente usare Naive Bayes e mentre IBk ottiene complessivamente metriche migliori, per Random Forest si tende a registrare un costo più basso. **Una eventuale scelta comunque, ricadrebbe su IBk, con la sola applicazione di balancing (SMOTE)**, dato che l'F1-score è il migliore, mentre kappa e AUC non sono le migliori, ma nemmeno le peggiori (si preferisce un F1-score più alto). La scelta di IBk con la sola applicazione di SMOTE è confermata da NPofB20. C'è comunque da ricordare che i dati, e il numero di iterazioni di walk forward per BookKeeper è basso, a causa delle poche release a disposizione.
- In Storm, conviene sempre scartare Naive Bayes, e per quasi tutte le metriche, usare IBk o Random forest con la configurazione: feature selection (best first backward), niente balancing e cost sensitivity (sensitive learning). Mantenendo la preferenza per un F1-score alto, ma anche tenendo in considerazione kappa, AUC e costi, **conviene sicuramente utilizzare IBk con feature selection (best first backward), no balancing e con cost sensitivity (sensitive learning)**, che rappresenta un buon compromesso per le metriche e il costo considerati. NPofB20 tende a favorire Random forest (si feature selection, no balancing, no sens. learning), ma AUC e Kappa e la poca differenza per l'F1-score, sposta l'ago della bilancia a favore di IBk con le tecniche scelte. I dati a disposizione e il numero di release elevati ha sicuramente permesso di ottenere dei risultati più affidabili rispetto a BookKeeper.

Minacce alla validità

- Si assume che le informazioni prelevate da Jira siano affidabili
- Le release e i ticket senza commit vengono scartati

Links

- GitHub: <https://github.com/StefanoBelli/isw2-deliverable>
- SonarCloud: https://sonarcloud.io/project/overview?id=StefanoBelli_isw2-deliverable

Grazie per l'attenzione!